JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DoubleBuffer

java.lang.Object
    java.nio.Buffer
        java.nio.DoubleBuffer

All Implemented Interfaces:

`Comparable<DoubleBuffer>`

```
public abstract sealed class DoubleBuffer
extends Buffer
implements Comparable<DoubleBuffer>
```

A double buffer.

This class defines four categories of operations upon double buffers:

- Absolute and relative *`get`* and *`put`* methods that read and write single doubles;
- Absolute and relative *`bulk get`* methods that transfer contiguous sequences of doubles from this buffer into an array;
- Absolute and relative *`bulk put`* methods that transfer contiguous sequences of doubles from a double array or some other double buffer into this buffer;
- A method for `compacting` a double buffer.

Double buffers can be created either by *`allocation`*, which allocates space for the buffer's content, by *`wrapping`* an existing double array into a buffer, or by creating a *view* of an existing byte buffer.

Like a byte buffer, a double buffer is either *direct* or *non-direct*. A double buffer created via the `wrap` methods of this class will be non-direct. A double buffer created as a view of a byte buffer will be direct if, and only if, the byte buffer itself is direct. Whether or not a double buffer is direct may be determined by invoking the `isDirect` method.

Methods in this class that do not otherwise have a value to return are specified to return the buffer upon which they are invoked. This allows method invocations to be chained.

Optional operations

Methods specified as *optional operations* throw a ReadOnlyBufferException when invoked on a read-only DoubleBuffer. The methods array and arrayOffset throw an UnsupportedOperationException if the DoubleBuffer is not backed by

an accessible double array (irrespective of whether the DoubleBuffer is read-only).

Since:

1.4

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
static DoubleBuffer	**allocate**(int capacity)	Allocates a new double buffer.
final double[]	**array**()	Returns the double array that backs this buffer *(optional operation)*.
final int	**arrayOffset**()	Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.
abstract DoubleBuffer	**asReadOnlyBuffer**()	Creates a new, read-only double buffer that shares this buffer's content.
final DoubleBuffer	**clear**()	Clears this buffer.
abstract DoubleBuffer	**compact**()	Compacts this buffer *(optional operation)*.
int	**compareTo**(DoubleBuffer that)	Compares this buffer to another.
abstract DoubleBuffer	**duplicate**()	Creates a new double buffer that shares this buffer's content.
boolean	**equals**(Object ob)	Tells whether or not this buffer is equal to another object.

final DoubleBuffer	**flip**()	Flips this buffer.
abstract double	**get**()	Relative *get* method.
DoubleBuffer	**get**(double[] dst)	Relative bulk *get* method.
DoubleBuffer	**get**(double[] dst, int offset, int length)	Relative bulk *get* method.
abstract double	**get**(int index)	Absolute *get* method.
DoubleBuffer	**get**(int index, double[] dst)	Absolute bulk *get* method.
DoubleBuffer	**get**(int index, double[] dst, int offset, int length)	Absolute bulk *get* method.
final boolean	**hasArray**()	Tells whether or not this buffer is backed by an accessible double array.
int	**hashCode**()	Returns the current hash code of this buffer.
abstract boolean	**isDirect**()	Tells whether or not this double buffer is direct.
final DoubleBuffer	**limit**(int newLimit)	Sets this buffer's limit.
final DoubleBuffer	**mark**()	Sets this buffer's mark at its position.
int	**mismatch**(DoubleBuffer that)	Finds and returns the relative index of the first mismatch between this buffer and a given buffer.
abstract ByteOrder	**order**()	Retrieves this buffer's byte order.
final DoubleBuffer	**position**(int newPosition)	Sets this buffer's position.
abstract DoubleBuffer	**put**(double d)	Relative *put* method *(optional operation)*.

final DoubleBuffer	**put**(double[] src)	Relative bulk *put* method *(optional operation)*.
DoubleBuffer	**put**(double[] src, int offset, int length)	Relative bulk *put* method *(optional operation)*.
abstract DoubleBuffer	**put**(int index, double d)	Absolute *put* method *(optional operation)*.
DoubleBuffer	**put**(int index, double[] src)	Absolute bulk *put* method *(optional operation)*.
DoubleBuffer	**put**(int index, double[] src, int offset, int length)	Absolute bulk *put* method *(optional operation)*.
DoubleBuffer	**put**(int index, DoubleBuffer src, int offset, int length)	Absolute bulk *put* method *(optional operation)*.
DoubleBuffer	**put**(DoubleBuffer src)	Relative bulk *put* method *(optional operation)*.
final DoubleBuffer	**reset**()	Resets this buffer's position to the previously-marked position.
final DoubleBuffer	**rewind**()	Rewinds this buffer.
abstract DoubleBuffer	**slice**()	Creates a new double buffer whose content is a shared subsequence of this buffer's content.
abstract DoubleBuffer	**slice**(int index, int length)	Creates a new double buffer whose content is a shared subsequence of this buffer's content.
String	**toString**()	

		Returns a string summarizing the state of this buffer.
static DoubleBuffer	**wrap**(double[] array)	Wraps a double array into a buffer.
static DoubleBuffer	**wrap**(double[] array, int offset, int length)	Wraps a double array into a buffer.

Methods declared in class Buffer

capacity, hasRemaining, isReadOnly, limit, position, remaining

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

allocate

```
public static DoubleBuffer allocate(int capacity)
```

Allocates a new double buffer.

The new buffer's position will be zero, its limit will be its capacity, its mark will be undefined, each of its elements will be initialized to zero, and its byte order will be the native order of the underlying hardware. It will have a backing array, and its array offset will be zero.

Parameters:

`capacity` - The new buffer's capacity, in doubles

Returns:

The new double buffer

Throws:

`IllegalArgumentException` - If the `capacity` is a negative integer

wrap

```
public static DoubleBuffer wrap(double[] array,
                                int offset,
                                int length)
```

Wraps a double array into a buffer.

The new buffer will be backed by the given double array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity will be `array.length`, its position will be `offset`, its limit will be `offset + length`, its mark will be undefined, and its byte order will be the `native order` of the underlying hardware. Its `backing array` will be the given array, and its `array offset` will be zero.

Parameters:

`array` - The array that will back the new buffer

`offset` - The offset of the subarray to be used; must be non-negative and no larger than `array.length`. The new buffer's position will be set to this value.

`length` - The length of the subarray to be used; must be non-negative and no larger than `array.length - offset`. The new buffer's limit will be set to `offset + length`.

Returns:

The new double buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

wrap

```
public static DoubleBuffer wrap(double[] array)
```

Wraps a double array into a buffer.

The new buffer will be backed by the given double array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity and limit will be `array.length`, its position will be zero, its mark will be undefined, and its byte order will be the `native order` of the underlying hardware. Its `backing array` will be the given array, and its `array offset` will be zero.

Parameters:

`array` - The array that will back this buffer

Returns:

The new double buffer

slice

```
public abstract DoubleBuffer slice()
```

Creates a new double buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of doubles remaining in this buffer, its mark will be undefined, and its byte order will be identical to that of this buffer. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

`slice` in class `Buffer`

Returns:

The new double buffer

slice

```
public abstract DoubleBuffer slice(int index,
                                   int length)
```

Creates a new double buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at position `index` in this buffer, and will contain `length` elements. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be `length`, its mark will be undefined, and its byte order will be identical to that of this buffer. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

`slice` in class `Buffer`

Parameters:

`index` - The position in this buffer at which the content of the new buffer will start; must be non-negative and no larger than `limit()`

`length` - The number of elements the new buffer will contain; must be non-negative and no larger than `limit() - index`

Returns:

The new buffer

Throws:

IndexOutOfBoundsException - If index is negative or greater than limit(), length is negative, or length > limit() - index

Since:

13

duplicate

```
public abstract DoubleBuffer duplicate()
```

Creates a new double buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, mark values, and byte order will be identical to those of this buffer. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

duplicate in class Buffer

Returns:

The new double buffer

asReadOnlyBuffer

```
public abstract DoubleBuffer asReadOnlyBuffer()
```

Creates a new, read-only double buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer; the new buffer itself, however, will be read-only and will not allow the shared content to be modified. The two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, mark values, and byte order will be identical to those of this buffer.

If this buffer is itself read-only then this method behaves in exactly the same way as the duplicate method.

Returns:

The new, read-only double buffer

get

```
public abstract double get()
```

Relative *get* method. Reads the double at this buffer's current position, and then increments the position.

Returns:

The double at the buffer's current position

Throws:

`BufferUnderflowException` - If the buffer's current position is not smaller than its limit

put

```
public abstract DoubleBuffer put(double d)
```

Relative *put* method *(optional operation)*.

Writes the given double into this buffer at the current position, and then increments the position.

Parameters:

`d` - The double to be written

Returns:

This buffer

Throws:

`BufferOverflowException` - If this buffer's current position is not smaller than its limit

`ReadOnlyBufferException` - If this buffer is read-only

get

```
public abstract double get(int index)
```

Absolute *get* method. Reads the double at the given index.

Parameters:

`index` - The index from which the double will be read

Returns:

The double at the given index

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit

put

```
public abstract DoubleBuffer put(int index,
                                 double d)
```

Absolute *put* method *(optional operation)*.

Writes the given double into this buffer at the given index.

Parameters:

`index` - The index at which the double will be written

`d` - The double value to be written

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative or not smaller than the buffer's limit

`ReadOnlyBufferException` - If this buffer is read-only

get

```
public DoubleBuffer get(double[] dst,
                        int offset,
                        int length)
```

Relative bulk *get* method.

This method transfers doubles from this buffer into the given destination array. If there are fewer doubles remaining in the buffer than are required to satisfy the request, that is, if `length > remaining()`, then no doubles are transferred and a `BufferUnderflowException` is thrown.

Otherwise, this method copies `length` doubles from this buffer into the given array, starting at the current position of this buffer and at the given offset in the array. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `src.get(dst, off, len)` has exactly the same effect as the loop

```
    for (int i = off; i < off + len; i++)
        dst[i] = src.get();
```

except that it first checks that there are sufficient doubles in this buffer and it is potentially much more efficient.

Parameters:

`dst` - The array into which doubles are to be written

`offset` - The offset within the array of the first double to be written; must be non-negative and no larger than `dst.length`

`length` - The maximum number of doubles to be written to the given array; must be non-negative and no larger than `dst.length - offset`

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than `length` doubles remaining in this buffer

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

get

```
public DoubleBuffer get(double[] dst)
```

Relative bulk *get* method.

This method transfers doubles from this buffer into the given destination array. An invocation of this method of the form `src.get(a)` behaves in exactly the same way as the invocation

```
    src.get(a, 0, a.length)
```

Parameters:

`dst` - The destination array

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than `length` doubles remaining in this buffer

get

```
public DoubleBuffer get(int index,
                        double[] dst,
                        int offset,
                        int length)
```

Absolute bulk *get* method.

This method transfers `length` doubles from this buffer into the given array, starting at the given index in this buffer and at the given offset in the array. The position of this buffer is unchanged.

An invocation of this method of the form `src.get(index, dst, offset, length)` has exactly the same effect as the following loop except that it first checks the consistency of the supplied parameters and it is potentially much more efficient:

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst[i] = src.get(j);
```

Parameters:

`index` - The index in this buffer from which the first double will be read; must be non-negative and less than `limit()`

`dst` - The destination array

`offset` - The offset within the array of the first double to be written; must be non-negative and less than `dst.length`

`length` - The number of doubles to be written to the given array; must be non-negative and no larger than the smaller of `limit() - index` and `dst.length - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

Since:

13

get

```
public DoubleBuffer get(int index,
                        double[] dst)
```

Absolute bulk *get* method.

This method transfers doubles from this buffer into the given destination array. The position of this buffer is unchanged. An invocation of this method of the form `src.get(index, dst)` behaves in exactly the same way as the invocation:

```
    src.get(index, dst, 0, dst.length)
```

Parameters:

`index` - The index in this buffer from which the first double will be read; must be non-negative and less than `limit()`

`dst` - The destination array

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative, not smaller than `limit()`, or `limit() - index < dst.length`

Since:

13

put

`public DoubleBuffer put(DoubleBuffer src)`

Relative bulk *put* method *(optional operation)*.

This method transfers the doubles remaining in the given source buffer into this buffer. If there are more doubles remaining in the source buffer than in this buffer, that is, if `src.remaining() > remaining()`, then no doubles are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies *n* = `src.remaining()` doubles from the given buffer into this buffer, starting at each buffer's current position. The positions of both buffers are then incremented by *n*.

In other words, an invocation of this method of the form `dst.put(src)` has exactly the same effect as the loop

```
    while (src.hasRemaining())
        dst.put(src.get());
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient. If this buffer and the source buffer share the same backing array or memory, then the result will be as if the source elements were first copied to an intermediate location before being written into this buffer.

Parameters:

`src` - The source buffer from which doubles are to be read; must not be this buffer

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer for the remaining doubles in the source buffer

`IllegalArgumentException` - If the source buffer is this buffer

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public DoubleBuffer put(int index,
                        DoubleBuffer src,
                        int offset,
                        int length)
```

Absolute bulk *put* method *(optional operation)*.

This method transfers `length` doubles into this buffer from the given source buffer, starting at the given `offset` in the source buffer and the given `index` in this buffer. The positions of both buffers are unchanged.

In other words, an invocation of this method of the form `dst.put(index, src, offset, length)` has exactly the same effect as the loop

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst.put(j, src.get(i));
```

except that it first checks the consistency of the supplied parameters and it is potentially much more efficient. If this buffer and the source buffer share the same backing array or memory, then the result will be as if the source elements were first copied to an intermediate location before being written into this buffer.

Parameters:

`index` - The index in this buffer at which the first double will be written; must be non-negative and less than `limit()`

`src` - The buffer from which doubles are to be read

`offset` - The index within the source buffer of the first double to be read; must be non-negative and less than `src.limit()`

`length` - The number of doubles to be read from the given buffer; must be non-negative and no larger than the smaller of `limit() - index` and `src.limit() - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

Since:

16

put

```
public DoubleBuffer put(double[] src,
                        int offset,
                        int length)
```

Relative bulk *put* method *(optional operation)*.

This method transfers doubles into this buffer from the given source array. If there are more doubles to be copied from the array than remain in this buffer, that is, if `length` > `remaining()`, then no doubles are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies `length` doubles from the given array into this buffer, starting at the given offset in the array and at the current position of this buffer. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `dst.put(src, off, len)` has exactly the same effect as the loop

```
    for (int i = off; i < off + len; i++)
        dst.put(src[i]);
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient.

Parameters:

`src` - The array from which doubles are to be read

`offset` - The offset within the array of the first double to be read; must be non-negative and no larger than `src.length`

`length` - The number of doubles to be read from the given array; must be non-negative and no larger than `src.length - offset`

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public final DoubleBuffer put(double[] src)
```

Relative bulk *put* method *(optional operation)*.

This method transfers the entire content of the given source double array into this buffer. An invocation of this method of the form `dst.put(a)` behaves in exactly the same way as the invocation

```
    dst.put(a, 0, a.length)
```

Parameters:

`src` - The source array

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public DoubleBuffer put(int index,
                        double[] src,
                        int offset,
                        int length)
```

Absolute bulk *put* method *(optional operation)*.

This method transfers `length` doubles from the given array, starting at the given offset in the array and at the given index in this buffer. The position of this buffer is unchanged.

An invocation of this method of the form `dst.put(index, src, offset, length)` has exactly the same effect as the following loop except that it first checks the consistency of the supplied parameters and it is potentially much more efficient:

```
    for (int i = offset, j = index; i < offset + length; i++, j++)
        dst.put(j, src[i]);
```

Parameters:

`index` - The index in this buffer at which the first double will be written; must be non-negative and less than `limit()`

`src` - The array from which doubles are to be read

`offset` - The offset within the array of the first double to be read; must be non-negative and less than `src.length`

`length` - The number of doubles to be read from the given array; must be non-negative and no larger than the smaller of `limit() - index` and `src.length - offset`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `index`, `offset`, and `length` parameters do not hold

`ReadOnlyBufferException` - If this buffer is read-only

Since:

13

put

```
public DoubleBuffer put(int index,
                        double[] src)
```

Absolute bulk *put* method *(optional operation)*.

This method copies doubles into this buffer from the given source array. The position of this buffer is unchanged. An invocation of this method of the form `dst.put(index, src)` behaves in exactly the same way as the invocation:

```
    dst.put(index, src, 0, src.length);
```

Parameters:

`index` - The index in this buffer at which the first double will be written; must be non-negative and less than `limit()`

`src` - The array from which doubles are to be read

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - If `index` is negative, not smaller than `limit()`, or `limit() - index < src.length`

`ReadOnlyBufferException` - If this buffer is read-only

Since:

13

hasArray

```
public final boolean hasArray()
```

Tells whether or not this buffer is backed by an accessible double array.

If this method returns `true` then the `array` and `arrayOffset` methods may safely be invoked.

Specified by:

`hasArray` in class `Buffer`

Returns:

`true` if, and only if, this buffer is backed by an array and is not read-only

array

```
public final double[] array()
```

Returns the double array that backs this buffer *(optional operation)*.

Modifications to this buffer's content will cause the returned array's content to be modified, and vice versa.

Invoke the `hasArray` method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

`array` in class `Buffer`

Returns:

The array that backs this buffer

Throws:

`ReadOnlyBufferException` - If this buffer is backed by an array but is read-only

`UnsupportedOperationException` - If this buffer is not backed by an accessible array

arrayOffset

```
public final int arrayOffset()
```

Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.

If this buffer is backed by an array then buffer position *p* corresponds to array index *p* + `arrayOffset()`.

Invoke the `hasArray` method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

`arrayOffset` in class `Buffer`

Returns:

The offset within this buffer's array of the first element of the buffer

Throws:

`ReadOnlyBufferException` - If this buffer is backed by an array but is read-only

`UnsupportedOperationException` - If this buffer is not backed by an accessible array

position

```
public final DoubleBuffer position(int newPosition)
```

Sets this buffer's position. If the mark is defined and larger than the new position then it is discarded.

Overrides:

`position` in class `Buffer`

Parameters:

`newPosition` - The new position value; must be non-negative and no larger than the current limit

Returns:

This buffer

Since:

9

limit

```
public final DoubleBuffer limit(int newLimit)
```

Sets this buffer's limit. If the position is larger than the new limit then it is set to the new limit. If the mark is defined and larger than the new limit then it is discarded.

Overrides:

`limit` in class `Buffer`

Parameters:

`newLimit` - The new limit value; must be non-negative and no larger than this buffer's capacity

Returns:

This buffer

Since:

9

mark

```
public final DoubleBuffer mark()
```

Sets this buffer's mark at its position.

Overrides:

`mark` in class `Buffer`

Returns:

This buffer

Since:

9

reset

```
public final DoubleBuffer reset()
```

Resets this buffer's position to the previously-marked position.

Invoking this method neither changes nor discards the mark's value.

Overrides:

`reset` in class `Buffer`

Returns:

This buffer

Since:

9

clear

```
public final DoubleBuffer clear()
```

Clears this buffer. The position is set to zero, the limit is set to the capacity, and the mark is discarded.

Invoke this method before using a sequence of channel-read or *put* operations to fill this buffer. For example:

```
    buf.clear();     // Prepare buffer for reading
    in.read(buf);    // Read data
```

This method does not actually erase the data in the buffer, but it is named as if it did because it will most often be used in situations in which that might as well be the case.

Overrides:

`clear` in class `Buffer`

Returns:

This buffer

Since:

9

flip

```
public final DoubleBuffer flip()
```

Flips this buffer. The limit is set to the current position and then the position is set to zero. If the mark is defined then it is discarded.

After a sequence of channel-read or *put* operations, invoke this method to prepare for a sequence of channel-write or relative *get* operations. For example:

```
    buf.put(magic);    // Prepend header
    in.read(buf);      // Read data into rest of buffer
    buf.flip();        // Flip buffer
    out.write(buf);    // Write header + data to channel
```

This method is often used in conjunction with the `compact` method when transferring data from one place to another.

Overrides:

`flip` in class `Buffer`

Returns:

This buffer

Since:

9

rewind

```
public final DoubleBuffer rewind()
```

Rewinds this buffer. The position is set to zero and the mark is discarded.

Invoke this method before a sequence of channel-write or *get* operations, assuming that the limit has already been set appropriately. For example:

```
    out.write(buf);    // Write remaining data
    buf.rewind();      // Rewind buffer
    buf.get(array);    // Copy data into array
```

Overrides:

`rewind` in class `Buffer`

Returns:

This buffer

Since:

9

compact

```
public abstract DoubleBuffer compact()
```

Compacts this buffer *(optional operation)*.

The doubles between the buffer's current position and its limit, if any, are copied to the beginning of the buffer. That is, the double at index p = `position()` is copied to index zero, the double at index p + 1 is copied to index one, and so forth until the double at index `limit()` - 1 is copied to index n = `limit()` - 1 - p. The buffer's position is then set to $n+1$ and its limit is set to its capacity. The mark, if defined, is discarded.

The buffer's position is set to the number of doubles copied, rather than to zero, so that an invocation of this method can be followed immediately by an invocation of another relative *put* method.

Returns:

This buffer

Throws:

ReadOnlyBufferException - If this buffer is read-only

isDirect

```
public abstract boolean isDirect()
```

Tells whether or not this double buffer is direct.

Specified by:

isDirect in class Buffer

Returns:

true if, and only if, this buffer is direct

toString

```
public String toString()
```

Returns a string summarizing the state of this buffer.

Overrides:

toString in class Object

Returns:

A summary string

hashCode

```
public int hashCode()
```

Returns the current hash code of this buffer.

The hash code of a double buffer depends only upon its remaining elements; that is, upon the elements from position() up to, and including, the element at limit() - 1.

Because buffer hash codes are content-dependent, it is inadvisable to use buffers as keys in hash maps or similar data structures unless it is known that their contents will not change.

Overrides:

hashCode in class Object

Returns:

The current hash code of this buffer

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(Object)

equals

`public boolean equals(Object ob)`

Tells whether or not this buffer is equal to another object.

Two double buffers are equal if, and only if,

1. They have the same element type,
2. They have the same number of remaining elements, and
3. The two sequences of remaining elements, considered independently of their starting positions, are pointwise equal. This method considers two double elements `a` and `b` to be equal if `(a == b) || (Double.isNaN(a) && Double.isNaN(b))`. The values `-0.0` and `+0.0` are considered to be equal, unlike `Double.equals(Object)`.

A double buffer is not equal to any other type of object.

Overrides:

`equals` in class `Object`

Parameters:

`ob` - The object to which this buffer is to be compared

Returns:

`true` if, and only if, this buffer is equal to the given object

See Also:

`Object.hashCode()`, `HashMap`

compareTo

`public int compareTo(DoubleBuffer that)`

Compares this buffer to another.

Two double buffers are compared by comparing their sequences of remaining elements lexicographically, without regard to the starting position of each sequence within its corresponding buffer. Pairs of `double` elements are compared as if by invoking `Double.compare(double,double)`, except that `-0.0` and `0.0` are considered to be equal. `Double.NaN` is considered by this method to be equal to itself and greater than all other `double` values (including `Double.POSITIVE_INFINITY`).

A double buffer is not comparable to any other type of object.

Specified by:

`compareTo` in interface `Comparable<DoubleBuffer>`

Parameters:

`that` - the object to be compared.

Returns:

A negative integer, zero, or a positive integer as this buffer is less than, equal to, or greater than the given buffer

mismatch

```
public int mismatch(DoubleBuffer that)
```

Finds and returns the relative index of the first mismatch between this buffer and a given buffer. The index is relative to the `position` of each buffer and will be in the range of 0 (inclusive) up to the smaller of the `remaining` elements in each buffer (exclusive).

If the two buffers share a common prefix then the returned index is the length of the common prefix and it follows that there is a mismatch between the two buffers at that index within the respective buffers. If one buffer is a proper prefix of the other then the returned index is the smaller of the remaining elements in each buffer, and it follows that the index is only valid for the buffer with the larger number of remaining elements. Otherwise, there is no mismatch.

Parameters:

`that` - The byte buffer to be tested for a mismatch with this buffer

Returns:

The relative index of the first mismatch between this and the given buffer, otherwise -1 if no mismatch.

Since:

11

order

```
public abstract ByteOrder order()
```

Retrieves this buffer's byte order.

The byte order of a double buffer created by allocation or by wrapping an existing `double` array is the `native order` of the underlying hardware. The byte order of a double buffer created as a view of a byte buffer is that of the byte buffer at the moment that the view is created.

Returns:

This buffer's byte order

Report a bug or suggest an enhancement

For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.